LISTA DE EXERCÍCIOS PARA ESTUDO AMS-ADS – Disciplina: TAPWM – 10/2026

JAVASCRIPT – EVENTOS – ***FINAL AULAS 4 E 5***

1) Se o arquivo index.html for executado em um navegador, qual será o **texto mostrado no input do resultado, quando o botão for clicado**? `" 0 21 42 63 84 10"`

Arquivo: index.html

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
   <meta charset="UTF-8">
   <title>Exercício formulário com evento </title>
</head>

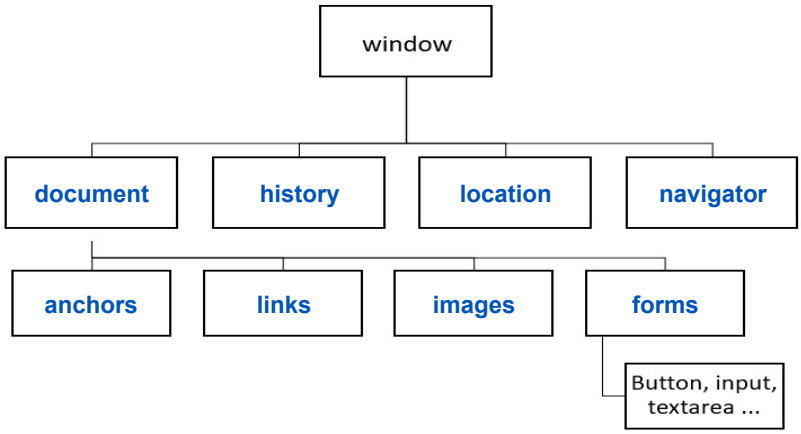
<body>
   <form id="formulario1">
      <br>
      <label>resultado</label>
      <input type="text" name="resultadoout" id="resultado" disabled>
      <br>
      <input type="button" value="Executa função" onclick="teste();">
   </form>
   <script>
      function teste() {
         const x = [2, 4, 6, 8, 10];
         let s = " ";
         x.forEach(f);
         console.log(s);

         function f(value, index, array) {
            s += index + " " + value; //observar espaço em branco
         }

         document.forms.formulario1.resultadoout.value = s;
      }
   </script>
</body>

</html>
```

2) Complete os quadros com os nomes dos **objetos da Hierarquia de objetos DOM** (Document Object Model) e **BOM** (Browser Object Model).



3) Supondo o código em um form chamado formulario1:

```
<input name="idEmail" type="email" id="idEmail">
```

Assinale **<u>todas</u>** as **formas possíveis formas para acessar o valor que for digitado no input**:

a) document.formulario1.elements[0].value
b) document.forms.formulario1.elements.idEmail.value

c) document.forms.formulario1.elements[0].value
d) document.getElementById("idEmail")
e) document.forms.formulario1.elements["idEmail"].value
f) document.getElementById("idEmail").value
g) document.getElementByName("idEmail")

4) Considere o código em JS abaixo:

```
<script>
 .... {
 this.nome = nome;
 }

 let cliente = new Cliente(“Paulo”);
 document.write(“o nome do cliente é:”+cliente.nome);
</script>
```

Para que o código funcione corretamente, **a parte ... deve preenchida com**:

a) Public class cliente(nome)
b) class cliente(nome)
c) let Cliente = function(nome)
d) public Cliente = function(nome)
e) default classe Cliente = function(nome)
f) function cliente(nome)

5) No código abaixo, se o **número de entrada for 4**, quando clicado o botão, o **resultado** mostrará: **24**

```
<body>
   <h1>Operação</h1>
   <form id="formulario1">
      <label>Escolha um número</label>
      <input type="number" id="numero">
      <br>
      <label>resultado</label>
      <input type="text" id="resultado" disabled>
      <br>
      <input type="button" value="Executa função" onclick="funcao3();">
   </form>
   <script>
      funcao1 = function(n) {
         return (n * 3);
      }
      funcao2 = function(n) {
         return funcao1(n * 2);
      }
      funcao3 = function() {
         document.forms.formulario1.elements[1].value =
            funcao2(document.forms.formulario1.elements[0].value);
      }
   </script>
</body>
```

6) Selecione a(s) opção(ões) que **NÃO** mostra(m) o **endereço da página atual** no JavaScript.

a) *window.location.href*
b) window.navigator.URL
c) *window.history.current*
d) *window.document.URL*

7) Considere o código:

```
<html>
 <input type='text' id='a' value='15'>
 <br>
 <input type='text' id='b' value='20'>
 <br>
 <input type='text' id='c'>
 <script>
        document.getElementById('c').value=(document.getElementById('a').value+
document.getElementById('b').value);
 </script>
 </html>
```

Quando executado irá representar **no navegador**:

a)

15
20
1520

b)

15
20
35

c)

15
20
0

d)

1520
1520
1520

a) Nenhuma das anteriores

JAVASCRIPT - NODE

8) Sobre a **definição e arquitetura fundamental do Node.js**, qual alternativa está **correta**?

a) Node.js é um framework JavaScript full-stack que opera sobre o motor SpiderMonkey da Mozilla.

b) Node.js é primariamente um ambiente de execução (*runtime*) construído sobre o motor V8 do Google Chrome para executar JavaScript fora do navegador.

c) O motor V8 compila o código JavaScript para linguagem de máquina, mas mantém a desvantagem de ser puramente interpretado durante a execução, o que limita sua performance.

d) A maior parte do "core" do Node.js, incluindo o motor V8 e as funcionalidades de I/O, é escrita em JavaScript para garantir alta portabilidade.

9) O modelo de **concorrência do Node.js** é um de seus maiores diferenciais. Qual característica **melhor** descreve esse modelo?

a) Ele se baseia em um modelo Multi-Thread, onde cada nova requisição cria uma nova thread do sistema operacional para ser tratada.

b) Utiliza um modelo de I/O bloqueante (síncrono), garantindo que a primeira requisição a entrar seja sempre a primeira a ser processada.

c) Usa um modelo de I/O direcionada a evento não bloqueante, operando em uma única thread principal (Event Loop), tornando-o eficiente para aplicações com troca intensa de dados.

d) O Node.js não possui desvantagens em tarefas intensivas em CPU, pois o V8 gerencia automaticamente o paralelismo em múltiplos núcleos.

10) Qual é a principal **função do NPM (Node Package Manager)** no ecossistema Node.js?
a) É um servidor web nativo utilizado para responder a requisições HTTP sem a necessidade de módulos externos.
b) É o motor que compila o código JavaScript para linguagem de máquina (V8).
c) É o gerenciador de pacotes padrão, essencial para manipulação de pacotes e dependências, facilitando a distribuição e compartilhamento de módulos.
d) É uma linguagem de template (view engine) utilizada para renderizar páginas HTML dinamicamente, como o EJS.

11) Popular framework web estruturado, escrito em JavaScript que roda sobre o ambiente Node.js. Ele fornece **API para controle de rotas** e permite **adicionar novos processos de requisição** por meio de "middleware" em qualquer ponto da "fila" de requisições. Trata-se de:
( ) EJS
( ) Express
( ) Nodemon
( ) Nenhuma das anteriores

12) Observe o trecho de código JavaScript utilizando o **framework Express.js** . Qual é a funcionalidade principal do **método app.get()**?

```
let express = require('express');
let app=express();

app.get('/informacao/historia', function(req,res){
   res.render("informacao/historia");
});

app.listen(3000, function(){ /* ... */ });
```

a) É o método responsável por iniciar o servidor web na porta 3000.
b) É o método utilizado para definir e responder a uma rota específica usando o verbo HTTP GET.
c) É o método que configura a view engine (motor de visualização) da aplicação para utilizar o EJS.
d) É o método usado para lidar com requisições do tipo POST enviadas por um formulário.

13) Qual afirmação define a **principal diferença entre o modelo de servidor tradicional (Thread-per-request) e o modelo Node.js (Event Loop)** e qual a **consequência** do modelo tradicional ao atingir o limite de recursos?
a) O modelo tradicional usa apenas uma thread e é totalmente assíncrono, enquanto o Node.js cria uma nova thread para cada requisição, mas consome menos memória.
b) O modelo tradicional cria uma nova thread para cada requisição (bloqueante), e ao atingir o limite, o servidor pode levar a uma falha de memória ou a requisições em espera (queueing).
c) O Node.js utiliza o modelo Multi-Thread para I/O, enquanto o modelo tradicional utiliza o Event Loop, resultando em desempenho superior no modelo tradicional.
d) A única diferença é a linguagem de programação usada, sendo o Node.js para JavaScript e o modelo tradicional para linguagens como PHP.

14) Qual **tipo de tarefa causa um problema de desempenho notável no Node.js devido à sua arquitetura single thread (Event Loop)**, e qual é a **solução** moderna nativa que mitiga esse problema?
a) Tarefas intensivas em I/O (Input/Output); a solução é usar o módulo http para desviar o processamento.
b) Tarefas intensivas em Rede; a solução é a biblioteca libuv que distribui a carga em threads secundárias de rede.
c) Tarefas intensivas em CPU (ex: cálculos pesados); a solução é o uso do módulo Worker Threads para mover o processamento para outras threads do sistema.
d) Tarefas que utilizam o NPM para instalação de pacotes; a solução é o uso do NPX para instalação temporária.

15) Analise o código de servidor HTTP abaixo. **Qual trecho de código JavaScript completa** corretamente o servidor para que ele responda **"Seu saldo é R$ 500,00" na rota /saldo e "Página Principal" para qualquer outra rota** (exemplo: /)?

```
let http = require('http');

let server = http.createServer( function(req,res){
    // ... CÓDIGO FALTANTE ...
} );

server.listen(3000);
```

a)

```
res.write("Seu saldo é R$ 500,00");
res.end();
```

b)

```
if(req.url === '/saldo') {
    res.end("Seu saldo é R$ 500,00");
} else {
    res.end("Página Principal");
}
```

c)

```
if(req.url == '/saldo') {
    res.end("Página Principal");
} else {
    res.end("Seu saldo é R$ 500,00");
}
```

d)

```
res.end(req.url == '/saldo' ? "Seu saldo é R$ 500,00" : "Erro 404");
```

16) Callback é um tipo de função que só é executada após o processamento de outra função. Na linguagem JavaScript, quando uma função é passada como um argumento de outra, ela é, então, chamada de **callback**. Considere o seguinte código:

```
function buscarDados(callback) {
  console.log("Buscando dados...");

  setTimeout(() => {
    const dados = { id: 1, nome: "Produto X" };
    callback(dados); // retorna os dados para o callback
  }, 2000);
}

buscarDados((resultado) => {
  console.log("Dados recebidos:", resultado);
});
```

Assinale a alternativa **incorreta**:

a) O setTimeout simula uma espera, como se estivesse buscando dados na internet.
b) callback(dados) entrega o resultado para quem chamou.
c) A função buscarDados inicia um temporizador (SetTImeout) de 2 segundos. Após o término desse tempo, o código dentro do temporizador chama o callback, imprimindo: "Dados recebidos: { id: 1, nome: 'Produto X' }".
d) A função callback aguarda 2 segundos, depois chama a função buscarDados para obter o resultado, imprimindo: "Dados recebidos: { id: 1, nome: 'Produto X' }"

17) À medida que o código cresce, especialmente com várias operações assíncronas em sequência, são criadas muitas callbacks, e o aninhamento de funções aumenta,

tornando o código difícil de ler e manter (Callback Hell). Para auxiliar nessa questão podem ser utilizadas as **Promises**, objetos que representam o resultado eventual de uma operação assíncrona (como uma requisição de rede, leitura de arquivo etc.). Elas podem estar em três estados: pendente (estado inicial), resolvida (sucesso) ou rejeitada (erro). Esses tipos de estruturas têm uma sintaxe mais limpa e fluida e tratam erros em um único lugar.

18) Qual é a principal vantagem do uso da sintaxe **async/await** em relação ao encadeamento de **Promises** (.then().catch()) para consumo de APIs?
a) O async/await executa as requisições em paralelo automaticamente, tornando-as mais rápidas que as Promises.
b) O async/await elimina completamente a necessidade de usar o bloco try/catch para tratamento de erros.
c) O async/await permite que o código assíncrono seja escrito de forma que se pareça e se comporte como código síncrono (sequencial), melhorando a legibilidade e a manutenção.
d) O async/await é um módulo nativo do Node.js, enquanto Promises requerem a instalação de um pacote NPM.

19) **Informe V(erdadeiro) ou F(also)** para as afirmativas:
(**V**) O Node.js é um ambiente de código aberto e multiplataforma, e é executado no mecanismo V8 JavaScript do Google.
(**V**) Embora o JavaScript seja usado tanto no front-end quanto no back-end com Node.js, a estrutura e as bibliotecas utilizadas em cada camada são geralmente diferentes.
(**F**) O Node.js é capaz de gerar conteúdos dinâmicos rodando JavaScript no servidor, porém não tem a capacidade de acessar banco de dados.
(**V**) Uma das principais características do Node.js é a programação assíncrona, o que permite lidar com várias conexões simultaneamente sem bloquear o processo principal.
(**V**) O NPM é o gerenciador de pacotes padrão do Node.js
(**F**) O V8 compila o código para linguagem de máquina, além de otimizar drasticamente a execução usando heurísticas (procedimentos, estratégias), permitindo que a execução seja feita em cima do código compilado e não interpretado.
(**F**) O Node.Js foi pensado para ser escalável, e se destaca pelo fato de trabalhar de forma síncrona utilizando todas as APIs nativas com I/O (input e output).

19 (6ª): F na leitura literal; o V8 também interpreta bytecode.

20) Considere o seguinte **trecho de código** ainda em desenvolvimento Node.js:

```
var express = require('express');
var app=express();
app.get('/', function(req,res){
    res.send("<html><body>Site Oficial – Home </body></html>");
});
app.get('/quemsomos', function(req,res){
   res.send("<html><body>Quem Somos?</body></html>");
});
app.get('/contato', function(req,res){
   res.send("<html><body>Faça aqui o seu contato</body></html>");
});
app.listen(3000, function(){
            console.log("servidor com express foi carregado");
});
```

a) Qual seriam os comandos que deverão ser **digitados no navegador para mostrar as páginas**:
- "Quem somos?" **http://localhost:3000/quemsomos**
- Home (página principal) **http://localhost:3000/**

b) Mostre um exemplo de comando a ser **digitado no navegador e que resultaria erro 404**: **http://localhost:3000/produtos**

GIT E GITHUB E SIMILARES (APRESENTAÇÕES)

21) O **Git** funciona como um sistema de controle de versão que rastreia alterações em arquivos, tirando "instantâneos" (commits) de cada estado do projeto.

22) Faça a **associação** dos conceitos:

a) repositório
b) git config
c) commit
d) versionamento (VCS)
e) merge
f) branch

(**d**) Sistema que gerencia as mudanças no código, permitindo voltar a estados anteriores, rastrear quem fez o quê e facilitar a colaboração.
(**a**) Estrutura em que o Git armazena o histórico de versões, os metadados e as informações necessárias para controlar as alterações do projeto.
(**c**) Um ponto de salvamento no histórico do projeto. Representa um snapshot (fotografia) das mudanças feitas desde o último commit.
(**f**) Uma linha de desenvolvimento independente que se separa da principal. Permite trabalhar em novas funcionalidades sem afetar o código estável.
(**e**) O processo de unir as alterações feitas em um Branch de volta a outra linha de desenvolvimento (ex: unir uma nova funcionalidade ao Branch principal).
(**b**) Comando de configuração inicial para definir a identidade do usuário (nome e e-mail) no ambiente Git local.

23) Considere as **ações abaixo realizadas em um repositório Git**:

O usuário criou um arquivo chamado index.html. Depois, digitou o comando: git add index.html Em seguida, digitou: git commit -m "Adiciona arquivo inicial"

Com base nessas informações, assinale a alternativa **correta**:
a) O comando git add envia o arquivo diretamente para o GitHub.
b) O comando git commit salva o arquivo no staging area.
c) O comando git add coloca o arquivo na staging area, preparando-o para o commit.
d) O comando git commit remove o arquivo do repositório.

24) Qual comando é usado para **verificar o estado atual do repositório** (arquivos modificados, adicionados ou não monitorados)?
a) git log
b) git status
c) git init
d) git clone

25**)** Qual comando mostra o **histórico de commits realizados**?
a) git history
b) git show
c) git log
d) git commit -h

26) Qual comando **envia commits para o repositório remoto no GitHub?**
a) git send
b) git push
c) git upload
d) git remote

27) Considere os comandos na sequência:

```
git add app.js

git commit -m "Atualiza lógica do app"

git push
```

O que vai acontecer quando **executar git push**?
a) O arquivo será apagado do repositório remoto.
b) O commit será enviado para o repositório remoto.
c) Apenas o arquivo app.js será enviado sem commit.
d) Nada, porque git push não envia commits.

28) Observe os seguintes comandos:

```
git branch novaFuncao

git checkout novaFuncao
```

Qual é o resultado dessa sequência?
a) Cria e entra na branch master.
b) Cria uma nova branch e muda para ela.
c) Deleta uma branch.
d) Copia arquivos da branch atual.

29) Você está trabalhando em um repositório Git e executa a seguinte **sequência de comandos:**

```
git checkout -b novaFuncao

git add script.js

git commit -m "Implementa nova função"

git checkout main

git merge novaFuncao
```

O que acontece quando o **comando git merge novaFuncao é executado**?
a) As alterações feitas na branch novaFuncao são mescladas (juntadas) com a branch main.
b) A branch novaFuncao é excluída automaticamente.
c) O Git envia a branch novaFuncao para o repositório remoto.
d) O Git cria uma nova branch chamada merge.

30) Observe a **sequência**:

```
Criado o arquivo inicial app.js com o código: console.log("Versão inicial");

git checkout main

Alterado arquivo app.js para: console.log("Mensagem da MAIN");

git add app.js
```

git commit -m "Atualiza mensagem na main"

git checkout -b novaFuncao

Alterado o mesmo arquivo app.js para: console.log("Mensagem da NOVA FUNCAO");

git add app.js

git commit -m "Atualiza mensagem na novaFuncao"

O que acontece ao **executar o comando: git merge novaFuncao**

a) O merge foi concluído automaticamente.
b) O Git descartou as alterações da branch novaFuncao.
c) O Git duplicou o arquivo app.js.
d) O Git gerou um conflito, pois as duas branches alteraram a mesma linha do arquivo.

Obs. 30: a, com ressalva - a própria branch já está atualizada (Already up to date).

31) Você está colaborando em um projeto hospedado em um repositório remoto. Durante o desenvolvimento, várias ações foram realizadas:

Um novo desenvolvedor entrou no time e copiou o repositório remoto para sua máquina usando o comando:

git clone https://servidor.com/projeto.git

Após fazer alterações locais, ele enviou suas modificações para o repositório remoto com:

git push

Outro membro da equipe sincronizou sua máquina com as atualizações do repositório usando:

git pull

Antes de aceitar as alterações no branch principal (main), o time abriu um Pull Request para revisar o código e permitir que a equipe fizesse comentários.

Esse Pull Request foi configurado para acionar automaticamente testes de integração contínua (CI).

Com base nessas informações, assinale a alternativa **correta**:

a) O comando git fetch baixa e já aplica automaticamente todas as alterações no código local.
b) O git push envia commits locais para o repositório remoto, enquanto git pull baixa e integra alterações do remoto no seu repositório local.
c) GitHub, GitLab e Bitbucket não suportam Pull/Merge Requests.
d) Pull Requests não permitem revisão de código, servem apenas para criar branches remotas.

32) Um desenvolvedor clonou um repositório do GitHub para seu computador, modificou alguns arquivos e executou os comandos git add . e git commit -m "Atualização do projeto". Entretanto, ao acessar o GitHub, percebeu que as alterações ainda não estavam disponíveis no repositório remoto.

a) **Explique** por que as alterações ainda não aparecem no GitHub.

Os comandos add e commit registram as alterações apenas localmente.
Falta executar push para enviá-las ao GitHub.

b) **Qual comando** o desenvolvedor deve executar para enviar os commits locais para a branch main do repositório remoto chamado origin?

git push origin main

c) **Explique a diferença** entre os comandos git fetch origin e git pull origin main.

git fetch origin baixa atualizações e atualiza as referências remotas,
sem integrá-las à branch local. git pull origin main baixa a main remota
e a integra à branch local atual, por merge ou rebase.

33) Uma equipe de desenvolvimento está escolhendo uma plataforma para hospedar seus repositórios Git e colaborar em projetos de software. A equipe deseja utilizar uma plataforma que ofereça integração nativa com ferramentas de gerenciamento de

projetos, especialmente o Jira, permitindo acompanhar branches, Pull Requests e implantações diretamente nas tarefas.

Qual **plataforma atende melhor** a essa necessidade?

a) GitHub, por ser voltado exclusivamente a projetos de código aberto e possuir integração nativa com o Jira.
b) GitLab, por oferecer uma plataforma integrada para todas as etapas do desenvolvimento e ser reconhecido pela automação CI/CD.
c) Bitbucket, por possuir integração nativa com o Jira e outras ferramentas do ecossistema Atlassian.
d) GitHub, por permitir apenas o desenvolvimento individual, sem recursos de colaboração em equipe.
e) Bitbucket, por ser uma plataforma exclusiva para projetos de código aberto, sem integração com ferramentas externas.

SEO - (APRESENTAÇÕES)

34) Uma loja virtual de tênis possui uma página com o title “Home | Loja”, o H1 “BEM-VINDO” e um texto que repete a expressão “tênis de corrida barato” dezenas de vezes. Ao analisar a página sob a ótica do SEO On-Page, o desenvolvedor decide corrigi-la. Qual **conjunto de ações** é o mais adequado?

a) Aumentar ainda mais a repetição de “tênis de corrida barato” no texto até atingir a densidade ideal de 5%, mantendo o title genérico para não confundir os robôs de busca.
b) Identificar a intenção de busca do público, reescrever o title e o H1 com o tema específico, organizar o conteúdo com H2 e H3 que respondam às dúvidas do leitor e usar o termo de forma natural, com sinônimos e termos relacionados.
c) Manter o title e o H1 como estão, adicionar um segundo H1 com a palavra-chave principal e criar uma lista de tags repetidas no final da página para reforçar o tema.
d) Substituir os títulos por elementos estilizados com CSS, já que o buscador avalia o design visual da página, e trocar os links por “clique aqui” para incentivar a navegação.
e) Concentrar os esforços em conseguir links de outros sites, pois o conteúdo e a estrutura da página não influenciam o ranqueamento.

35) Uma equipe de desenvolvimento criou duas versões de uma página de loja virtual. As duas têm aparência visual praticamente idêntica, pois usam estilos CSS semelhantes. Na versão A, o cabeçalho, o título principal e os produtos foram marcados apenas com , e , e cada produto traz uma imagem sem alt e um link com o texto “Clique aqui”. Na versão B, a página usa <header>, <nav>, <main>, <h1>, <section> com <h2>, <article> com <h3>, imagens com alt descritivo e links como “Ver detalhes do notebook gamer”.

Com base nos fundamentos de SEO On-Page e HTML semântico, qual afirmação **melhor** descreve a comparação entre as duas versões?

a) As duas versões têm a mesma qualidade, pois a aparência visual, controlada pelo CSS, é o que determina a qualidade da marcação HTML.
b) A versão B garante a primeira posição nos resultados do Google, pois o uso de elementos semânticos é um fator que assegura melhor classificação.
c) A versão B é superior em estrutura, acessibilidade e clareza, pois explicita a organização do documento, oferece alternativas textuais às imagens e usa links com texto âncora informativo, ainda que isso não garanta posição específica nos resultados.

d) A versão A é mais adequada, pois elementos genéricos como são sempre preferíveis aos elementos semânticos e os níveis de heading devem ser escolhidos conforme o tamanho de fonte desejado.
e) A versão B só seria melhor se as URLs dos produtos usassem apenas parâmetros, como /p?id=123, já que URLs descritivas prejudicam o SEO.

36) Com base em SEO Técnico e SEO de Conteúdo (On-Page), avalie as **afirmações** abaixo:

I. A intenção de busca é mais importante do que repetir palavras-chave, pois o Google prioriza o motivo real da pesquisa do usuário.
II. O Title Tag e a Meta Descrição são elementos fundamentais do SEO On-Page, aparecendo diretamente nos resultados de busca.
III. O Google utiliza a versão **mobile** do site para fins de ranqueamento (Mobile-First).
IV. URLs longas e cheias de parâmetros são consideradas melhores pelo Google, pois indicam grande quantidade de informação.

Assinale a alternativa **correta**:
a) Apenas as afirmações I e II estão corretas.
b) Apenas as afirmações I, II e III estão corretas.
c) Apenas as afirmações II, III e IV estão corretas.
d) Todas estão corretas.

37) Considere o seguinte trecho de código HTML:

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Loja Exemplo</title>
</head>
<body>

  <h1>Produtos</h1>

  <h1>Promoções da Semana</h1>

  <section>
    <h3>Novidades</h3>
    <p>Confira os lançamentos mais recentes.</p>
  </section>

  <img src="camiseta.jpg">

</body>
</html>
```

Com base nas boas práticas de **SEO On-Page**, **semântica HTML** e uso correto de **headings**, assinale a alternativa **correta**:
a) O código está correto, pois páginas podem ter vários <h1> sem prejuízo ao SEO.
b) A imagem deveria possuir um atributo alt, pois isso melhora acessibilidade e SEO.
c) O <h3> está correto mesmo sem existir um <h2> antes dele.
d) O ideal para SEO é não usar headings, apenas com classes.

38) Os Core Web Vitals são métricas utilizadas pelo Google para avaliar a qualidade da experiência do usuário em uma página. **Analise** as afirmações abaixo:

I. O LCP (Largest Contentful Paint) mede o tempo até o maior elemento visível da página (como um banner ou o título principal) ser exibido, e o valor considerado bom é de até 2,5 segundos.

II. O INP (Interaction to Next Paint) substituiu o FID (First Input Delay) em março de 2024. Enquanto o FID media apenas o atraso da primeira interação, o INP avalia a

responsividade de todas as interações durante a visita, e o valor considerado bom é de até 200 ms.

III. O CLS (Cumulative Layout Shift) avalia a estabilidade visual da página. Imagens sem os atributos width e height e anúncios inseridos sem espaço reservado são causas comuns de CLS alto, e o valor considerado bom é de até 0,1.

IV. Nos dados de campo (relatório CrUX), uma página é aprovada nos Core Web Vitals quando a média simples de todos os acessos atinge o nível “Bom” em pelo menos uma das três métricas.

Assinale a **alternativa correta**:

a) Apenas as afirmações I e III estão corretas.

b) Apenas as afirmações I, II e III estão corretas.

c) Apenas as afirmações II, III e IV estão corretas.

d) Todas as afirmações estão corretas.

39) O conceito **Mobile First** no contexto do SEO (Search Engine Optimization) é crucial para o ranqueamento de um website. Qual das alternativas abaixo **melhor** explica a estratégia "Mobile First Indexing" do Google e sua implicação no desenvolvimento web?

a) Mobile First significa que a versão desktop do site deve ter mais funcionalidades e conteúdo do que a versão mobile, pois o Google prioriza o ranqueamento do código mais robusto.
b) O Mobile First Indexing é a prática de garantir que o conteúdo, a velocidade e a usabilidade da versão móvel do site sejam os principais fatores considerados pelo Google para a classificação nas buscas, exigindo que o desenvolvimento comece pela experiência de tela pequena.
c) Mobile First é uma técnica de design que exige que todos os sites tenham um aplicativo nativo para Android e iOS, sendo que o SEO só é aplicado após o lançamento do app.
d) Essa estratégia exige que os desenvolvedores criem dois sites completamente separados (um para desktop e outro para mobile) e o Google ranqueia o site que tiver o maior número de *backlinks*.
e) O Google desconsidera totalmente a versão desktop e o Mobile First apenas exige que as imagens sejam menores na versão mobile, sem afetar o ranqueamento do conteúdo textual.

40) Informe na frente de cada afirmativa se ela se refere ao **Google Search Console (GSC)** ou ao **Google Analytics 4 (GA4)**.

Afirmativa	**Ferramenta**
Permite consultar cliques, impressões, taxa de cliques e posição média de um site nos resultados da Pesquisa Google.	**GSC**
Utiliza uma estrutura baseada em eventos, isto é, em interações mensuráveis, como a visualização de uma página ou a realização de uma ação importante.	**GA4**
Permite verificar a indexação das páginas, identificar dificuldades de rastreamento e enviar páginas para nova análise.	**GSC**

Afirmativa	Ferramenta
Permite identificar por quais canais os visitantes chegaram ao site, como pesquisa orgânica, acesso direto ou redes sociais.	**GA4**
Ajuda a entender a presença do site na Pesquisa Google **antes** do acesso, indicando, por exemplo, páginas com muitas impressões e poucos cliques.	**GSC**
Ajuda a entender o que ocorre **depois** que o visitante entra, como a jornada pelas páginas e em quais etapas as pessoas deixam de realizar uma ação esperada.	**GA4**